Project Report

Open Source Software

Aritra Dhuan

24BAI10513

Course Code & Slot : BL2025260500688 (D11)

29th March, 2026

Content

1	Introduction	2 - 3
2	Part A — Origin and Philosophy 2.1 Origin Story 2.2 License Analysis 2.3 Ethical Reflection	4 - 7
3	Part B — Linux Footprints	8 - 9
4	Part C — FOSS Ecosystem	10
5	Part D — Open Source vs Proprietary	11
6	Shell Scripts	12 - 23
7	Conclusion	24
8	References	25

Introduction

Today open-source software is really important for our world. A lot of the systems that make the internet and smartphones work and that help people make software are built using open source technology. Open source software is different from software because people can look at the code and change it which makes it more open and easier for people to work together.

One of the reasons open-source software is so successful is because it is made by a community of people. Developers from around the world help make the software better, fix problems and add new things. When people work together like this the software gets better, more reliable and safer over time. That is why open-source tools are used in schools and in industries.

This report is about Git, which's a tool that helps people working on software keep track of the changes they make. Git is used by a lot of people who make software because it helps them manage versions of their code and work together on big projects. It is an important tool in the software business and is used by individuals and big companies.

The goal of this project is to look at Git from different angles like where it came from, how it is licensed and what it means for ethics. We will also use shell scripts to show how Linux tools can be used to automate tasks and work with open-source software.

Through this report I want to learn more about the ideas behind open-source software. How they affect the way people make software

today. This study will look at the side of Git but also the ideas and values that are behind the open-source movement and open-source software.

Origin

Origin Story of Git

Managing large softwares, including multiple subsystems and multiple lines of codes,which needs multiple users to maintain it, always had a problem. When numerous contributors are working on the same codebase, it becomes delicate to track changes, resoluteness conflicts, and maintain thickness. Before ultramodern interpretation control systems came wide, inventors frequently faced issues similar as overwritten law, loss of former performances, and hamstrung collaboration.

This challenge was particularly significant during the development of the Linux kernel, which involved thousands of inventors contributing from different corridors of the world. To manage this complexity, the development platoon originally reckoned on a personal interpretation control system known as BitKeeper. While BitKeeper was effective, it was not open source, which means the inventors don't have full control over the tool they were using.

Over time, licensing conflicts arose between the Linux community and those from BitKeeper. As a result, access to BitKeeper was confined, leaving the Linux inventors without a dependable system to manage their law. This situation created a need for a new control system that was not only effective but also free from any sort of restrictions.

In response to this problem, Linus Torvalds developed Git in 2005. His goal was to produce a system that was presto, dependable, and able to handle large- scale development efficiently. Unlike traditional centralized systems, Git was designed as a distributed interpretation control system, meaning that every inventor would have a complete dupe of the design depository. This excluded reliance on a central garçon and bettered both speed and trustability.

Another important aspect of Git's design was its focus on open- source principles. By making Git open source, Linus Torvalds gave full freedom to the inventors to use, modify and distribute the software.

In my opinion, the creation of Git shows how open source solutions often come from real world problems. Instead of relying on restricted systems, developers built tools that matched their needs and values. Today, Git is one of the most widely used version control systems and plays a key role in modern software development.

License Analysis

Git is released under the GNU General Public License version 2 (GPL v2). It is a widely used open-source license that makes absolutely sure software is staying free to use, modify, and also share. This license is rooted in the concept of the four freedoms of free software, those being: the freedom to run the program for any purpose, study/modify it however you want, redistribute copies, and distribute your modified versions, too.

Git meets all of those four freedoms, which makes it a true open-source software. However the GPL v2 does include an important condition. Namely, that any modified version that gets distributed also has to be shared under the same license. You need to share the source code as well. This is what makes sure that the software stays open and cannot be turned into a closed, proprietary product.

A company could modify Git then sell it, but here is the catch, it has to provide the source code and cannot stop others from further modifying it or sharing. This separates it from something like the MIT license. The MIT License is more permissive, it lets developers make closed-source versions without sharing what they changed.

Git falls into the “free as in freedom” category. It gives users complete control of the software. It is more than just free access, and that is important.

Ethical Reflection

Open source software does raise some interesting ethical questions about how knowledge, and technology, probably should be shared, I think. It is my opinion that not all software must be open source, but there is a lot to consider. Open source definitely promotes transparency, learning, and collaboration, all of which are really valuable for innovation. But, some companies rely on proprietary software in order to protect their business models and their investments which is also understandable.

Another ethical concern that people bring up is whether or not it is entirely fair for companies to build profitable products basically on top of open-source software. I mean, a lot of large organizations use things like Linux and Git in their systems. And while that might seem like they are just benefiting from community efforts, they also often contribute back through different improvements, or by providing funding, or support. So it can easily be seen as a pretty balanced relationship as opposed to pure exploitation.

The whole idea of “standing on the shoulders of giants” is very relevant when we are talking about open source. Developers are able to build upon existing work, which helps instead of everyone starting from scratch, which really accelerates progress overall. In my view I believe open source actually supports innovation instead of limiting it because it allows more people just to contribute ideas and make improvements.

So, overall, open source creates more of a collaborative and accessible environment, and although a balance between total openness and just basic business needs is pretty important to consider.

Linux Footprints

Git is a command-line tool that integrates easily into a Linux environment. It can be installed using package managers such as `apt` with the command `sudo apt install git`. After installation, its version can be checked using `git --version`, confirming that it is properly set up.

```
aritradhuan@aritraVivobook:~$ sudo apt install git
[sudo: authenticate] Password:
git is already the newest version (1:2.51.0-1ubuntu1).
The following package was automatically installed and is no longer required:
  nvidia-firmware-580-580.95.05
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 32
aritradhuan@aritraVivobook:~$ git --version
git version 2.51.0
aritradhuan@aritraVivobook:~$
```

The main Git executable is usually located in `/usr/bin/git`, which is a standard directory for system binaries. Git configuration files are stored in different locations based on their scope. The global configuration file is found in the user's home directory as `~/.gitconfig`, while project-specific settings are stored inside the `.git` directory within each repository.

```
aritradhuan@aritraVivobook:~$ sudo apt install git
[sudo: authenticate] Password:
git is already the newest version (1:2.51.0-1ubuntu1).
The following package was automatically installed and is no longer required:
  nvidia-firmware-580-580.95.05
Use 'sudo apt autoremove' to remove it.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 32
aritradhuan@aritraVivobook:~$ git --version
git version 2.51.0
aritradhuan@aritraVivobook:~$ which git
/usr/bin/git
aritradhuan@aritraVivobook:~$
```

Unlike some software, Git does not run as a background service. It operates only when a user executes commands, making it lightweight and efficient. It runs under the current user account, which ensures that all actions follow user-level permissions and enhances security.

Updates for Git are handled through the Linux package manager. By running `sudo apt update` and `sudo apt upgrade`, users can receive the latest features and security patches. This reflects the continuous support and improvement provided by the open-source community.

FOSS Ecosystem

Git does not exist in isolation; it is part of a larger open-source ecosystem which supports modern software development practices. It depends on different other technologies, such as the C programming language because Git itself is written in C. It also makes use of tools like SSH for secure communication and OpenSSL for encryption, which enables safe data transfer between systems.

Git has also enabled the development of many platforms and tools built on top of it. Popular platforms like GitHub and GitLab provide web-based interfaces for Git repositories, making collaboration a lot easier for developers around the world. These platforms offer features like version tracking, issue management, and continuous integration, and other useful things.

In a larger environment, Git is playing a central role in software development workflows, without a doubt. It is commonly used alongside tools in development, and is essential for managing source code in both small and large projects. The open source community around Git is very active, with developers constantly contributing to its improvement, and maintaining its reliability and security.

In my personal opinion, Git acts as the foundation for many modern development tools, softwares, making its a crucial component for the open source ecosystem and it is a really important foundation for future developments.

Open Source vs Proprietary

Feature	Git (Open Source)	BitKeeper (Proprietary)
Cost	Free to use	Paid software
Security	Source code is open and can be audited	Source code is closed
Support	Community-driven support	Company-based support
Freedom to Modify	High — users can modify and share	Limited — modifications restricted
Control	Distributed system, no central authority	Centralized control by company

Shell Scripts

Script 1 — System Identity Report

Script

```
STUDENT_NAME="Aritra"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)

DISTRO=$(lsb_release -d | cut -f2)

echo "================================"
echo " Open Source Audit -- $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo "License  : GPL (Linux Kernel)"
```

Output

```
aritradhuan@vivobook:~$ ls
Desktop     Downloads     Hello.java   Pictures   Templates   google-chrome-stable_current_amd64.deb   snap
Documents   Hello.class   Music        Public     Videos      script1.sh                               upscayl
aritradhuan@vivobook:~$ chmod +x script1.sh
aritradhuan@vivobook:~$ ./script1.sh
================================
 Open Source Audit — Aritra
================================
Software : Git
Distro   : Ubuntu 25.10
Kernel   : 6.17.0-19-generic
User     : aritradhuan
Uptime   : up 5 days, 6 hours, 53 minutes
Date     : Sun Mar 29 17:33:48 IST 2026
License  : GPL (Linux Kernel)
aritradhuan@vivobook:~$
```

Explanation

This script displays system information such as kernel version, user, uptime, and Linux distribution. It uses variables, command substitution, and echo formatting.

Script 2 — FOSS Package Inspector

Script

```
PACKAGE="git"

if dpkg -l | grep -q $PACKAGE; then
    echo "$PACKAGE is installed."
    dpkg -l | grep $PACKAGE
else
    echo "$PACKAGE is NOT installed."
fi

case $PACKAGE in
    git) echo "Git: distributed version control for collaboration" ;;
    apache2) echo "Apache: web server powering the internet" ;;
    mysql) echo "MySQL: database system" ;;
    firefox) echo "Firefox: open-source web browser" ;;
esac
```

Output

```
aritradhuan@vivobook:~$ ls
Desktop   Downloads   Hello.java   Pictures   Templates   google-chrome-stable_current_amd64.deb   script2.sh   myscript
Documents   Hello.class   Music   Public   Videos   script1.sh   snap   zFav.txt
aritradhuan@vivobook:~$ chmod +x script2.sh
aritradhuan@vivobook:~$ ./script2.sh
git is installed.
ii  fonts-ubuntu                                    0.869+git20240321-0ubuntu1                 all          sans-serif font set from Ubuntu
ii  git                                             1:2.51.0-1ubuntu1                          amd64        fast, scalable, distributed revision control system
ii  git-man                                         1:2.51.0-1ubuntu1                          all          fast, scalable, distributed revision control system (manual pages)
ii  grub-common                                     2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader, version 2 (dummy package)
ii  grub-efi-amd64                                  2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader, version 2 (EFI-AMD64 version)
ii  grub-efi-amd64-bin                              2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader, version 2 (EFI-AMD64 modules)
ii  grub-efi-amd64-signed                           1.214+2.14~git20250718.0e36779-1ubuntu4    amd64        GRand Unified Bootloader, version 2 (EFI-AMD64 version, signed)
ii  grub-efi-amd64-unsigned                         2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader, version 2 (EFI-AMD64 images)
rc  grub-pc                                         2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader, version 2 (PC/BIOS version)
ii  grub2-common                                    2.14~git20250718.0e36779-1ubuntu4          amd64        GRand Unified Bootloader (common files for version 2)
ii  libcdio-cdda2t64:amd64                          10.2+2.0.2-1                               amd64        library to read and control digital audio CDs
ii  libcdio-paranoia2t64:amd64                      10.2+2.0.2-1                               amd64        library to read digital audio CDs with error correction
ii  libcpuinfo0:amd64                               0.0~git20250327.39ea79a-1                  amd64        CPU INFOrmation library (shared object)
ii  libdc1394-25:amd64                              2.2.6-5                                    amd64        high level programming interface for IEEE 1394 digital cameras
ii  libdv4t64:amd64                                 1.0.0-17.1build1                           amd64        software library for DV format digital video (runtime lib)
ii  libgphoto2-6t64:amd64                           2.5.31-4build1                             amd64        gphoto2 digital camera library
ii  libgphoto2-l10n                                 2.5.31-4build1                             all          gphoto2 digital camera library - localized messages
ii  libgphoto2-port12t64:amd64                      2.5.31-4build1                             amd64        gphoto2 digital camera port library
ii  libimobiledevice-1.0-6:amd64                    1.3.0+git20250228-2                        amd64        Library for communicating with iPhone and other Apple devices
ii  libjxr-tools                                    1.2~git20170615.f752187-5.3                amd64        JPEG-XR lib - command line apps
ii  libjxr0t64:amd64                                1.2~git20170615.f752187-5.3                amd64        JPEG-XR lib - libraries
ii  libldacbt-abr2:amd64                            2.0.2.3+git20200429+ed310a0-5              amd64        LDAC Bluetooth ABR library (shared library)
ii  libldacbt-enc2:amd64                            2.0.2.3+git20200429+ed310a0-5              amd64        LDAC Bluetooth encoder library (shared library)
ii  libplymouth5:amd64                              24.004.60+git20250831.4a3c171d-0ubuntu1    amd64        graphical boot animation and logger - shared libraries
ii  libpthreadpool0:amd64                           0.0~git20240616.560c60d-1                  amd64        pthread-based thread pool for C/C++ (shared object)
ii  librtmp1:amd64                                  2.4+20151223.gitfa8646d.1-2build8          amd64        toolkit for RTMP streams (shared library)
ii  libspatialaudio0t64:amd64                       0.3.0+git20180730+dfsg1-2.1build1          amd64        library for ambisonic encoding and decoding (runtime files)
ii  libwebp7:amd64                                  1.5.0-0.1                                  amd64        Lossy compression of digital photographic images (shared library)
ii  libwebpdemux2:amd64                             1.5.0-0.1                                  amd64        Lossy compression of digital photographic images (libwebpdemux library)
ii  libwebpmux3:amd64                               1.5.0-0.1                                  amd64        Lossy compression of digital photographic images (libwebpmux library)
ii  libx264-164:amd64                               2:0.164.3108+git31e19f9-2build2            amd64        x264 video coding library
ii  libxnnpack0.20241108:amd64                      0.0~git20241108.4ea82e5-2                  amd64        High-efficiency floating-point neural network inference operators (libs)
ii  linux-firmware                                  20250901.git993ff19b-0ubuntu1.7            all          Firmware for Linux kernel drivers
ii  lshw                                            02.19.git.2021.06.19.996aaad9c7-2.1ubuntu1 amd64        information about hardware configuration
ii  node-hosted-git-info                            6.1.1-2                                    all          Provides metadata from Github, Bitbucket and Gitlab
ii  node-ignore                                     5.2.1-1                                    all          manager and filter for .gitignore rules - Node.js library
ii  node-lcov-parse                                 1.0.0+20170612git80d039574ed9-6            all          Parse lcov results files and return JSON
ii  node-log-driver                                 1.2.7+git+20180219+bba1761737-7            all          simple logging framework in pure javascript
ii  plymouth                                        24.004.60+git20250831.4a3c171d-0ubuntu1    amd64        boot animation, logger and I/O multiplexer
ii  plymouth-label                                  24.004.60+git20250831.4a3c171d-0ubuntu1    amd64        boot animation, logger and I/O multiplexer - label control
ii  plymouth-theme-spinner                          24.004.60+git20250831.4a3c171d-0ubuntu1    amd64        boot animation, logger and I/O multiplexer - spinner theme
ii  plymouth-theme-ubuntu-text                      24.004.60+git20250831.4a3c171d-0ubuntu1    amd64        boot animation, logger and I/O multiplexer - ubuntu text theme
Git: distributed version control for collaboration
aritradhuan@vivobook:~$
```

Explanation

This script checks if Git is installed and prints its details. It uses if-else, grep, and case statements.

Script 3 — Disk & Permission Auditor

Script

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
```

Output

```
aritradhuan@vivobook:~$ ls
Desktop     Downloads    Hello.java  Pictures  Templates  google-chrome-stable_current_amd64.deb  script2.sh  snap
Documents   Hello.class  Music       Public    Videos     script1.sh                              script3.sh  upscayl
aritradhuan@vivobook:~$ chmod +x script3.sh
aritradhuan@vivobook:~$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 14M
/var/log => Permissions: drwxr-xr-x root syslog | Size: 461M
/home => Permissions: drwxr-xr-x root root | Size: 16G
/usr/bin => Permissions: drwxr-xr-x root root | Size: 382M
/tmp => Permissions: drwxrwxrwt root root | Size: 8.0K
aritradhuan@vivobook:~$
```

Explanation

This script checks directories, their size, and permissions. It uses loops, conditions, and awk.

Script 4 — Log File Analyzer

Script

```
LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "File not found"
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Output

```
aritradhuan@vivobook:~$ ls
Desktop   Documents   Hello.java   Pictures   Templates   google-chrome-stable_current_amd64.deb   script2.sh   script4.sh   snap
Downloads   Hello.class   Music   Public   Videos   script1.sh   script3.sh   snap   zFav.txt
aritradhuan@vivobook:~$ ./script4.sh /var/log/syslog error
Keyword 'error' found 1592 times
2026-03-29T17:48:12.448593+05:30 aritraVivobook google-chrome.desktop[86072]: [86128:86134:0329/174812.448444:ERROR:net/socket/ssl_client_socket_impl.cc:916] handshake faile
d; returned -1, SSL error code 1, net_error -202
2026-03-29T17:48:12.457424+05:30 aritraVivobook google-chrome.desktop[86072]: [86128:86134:0329/174812.457338:ERROR:net/socket/ssl_client_socket_impl.cc:916] handshake faile
d; returned -1, SSL error code 1, net_error -202
2026-03-29T17:48:15.038705+05:30 aritraVivobook google-chrome.desktop[86072]: [86128:86134:0329/174815.038319:ERROR:net/socket/ssl_client_socket_impl.cc:916] handshake faile
d; returned -1, SSL error code 1, net_error -202
2026-03-29T17:48:16.567465+05:30 aritraVivobook google-chrome.desktop[86072]: [86128:86134:0329/174816.566920:ERROR:net/socket/ssl_client_socket_impl.cc:916] handshake faile
d; returned -1, SSL error code 1, net_error -202
2026-03-29T17:48:16.588251+05:30 aritraVivobook google-chrome.desktop[86072]: [86128:86134:0329/174816.588117:ERROR:net/socket/ssl_client_socket_impl.cc:916] handshake faile
d; returned -1, SSL error code 1, net_error -202
aritradhuan@vivobook:~$
```

Explanation

This script is designed to look at a system log file and count how many times a particular keyword shows up. For testing purposes, this was done using `/var/log/syslog`, a typical system log, and the keyword being searched for was “error”.

Basically, the script needs the log file as input, and then it goes through it line by line, using a loop so it reads each line. As it is going through each line, it is checking to see if the keyword is there, and it is doing this without paying attention to capitalization. Meaning upper case or lower case do not matter. If it finds a match, it increases a counter.

Before doing anything else, the script also makes sure that the file you are trying to look at actually exists. If it cannot find the file, it will show an error message and stop. This is important; it helps to avoid problems that would come up while the script is running.

Once it has gone through the whole file, the script will tell you how many times it found the keyword “error.” It also shows you the last five lines, where the keyword appeared, which makes it easier to quickly spot any recent problems in the system logs.

In general, this script shows you many things, such as how to use loops, conditions, variables, and even command-line arguments using Bash. And showing how you are able to analyze log files when you are troubleshooting something.

Script 5 — Manifesto Generator

Script

```
echo "Answer the questions"

read -p "Tool you use: " TOOL
read -p "Freedom means: " FREEDOM
read -p "What will you build: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto.txt"

echo "On $DATE, I believe open source is about $FREEDOM." > $OUTPUT
echo "Tools like $TOOL help developers collaborate." >> $OUTPUT
echo "I would like to build $BUILD and share it freely." >> $OUTPUT

echo "Saved to $OUTPUT"
cat $OUTPUT
```

Output

```
aritradhuan@vivobook:~$ ls
Desktop     Downloads     Hello.java  Pictures  Templates  google-chrome-stable_current_amd64.deb  script2.sh  script4.sh
Documents   Hello.class   Music       Public    Videos     script1.sh                              script3.sh  script5.sh
aritradhuan@vivobook:~$ ./script5.sh
Answer the questions
Tool you use: Python
Freedom means: Everything
What will you build: Own LLM
Saved to manifesto.txt
On 29 March 2026, I believe open source is about Everything.
Tools like Python help developers collaborate.
I would like to build Own LLM and share it freely.
aritradhuan@vivobook:~$
```

opening the manifesto.txt file

```
aritradhuan@vivobook:~$ ls
Desktop     Downloads     Hello.java  Pictures  Templates  google-chrome-stable_current_amd64.deb  script2.sh  script4.sh
Documents   Hello.class   Music       Public    Videos     script1.sh                              script3.sh  script5.sh
aritradhuan@vivobook:~$ ./script5.sh
Answer the questions
Tool you use: Python
Freedom means: Everything
What will you build: Own LLM
Saved to manifesto.txt
On 29 March 2026, I believe open source is about Everything.
Tools like Python help developers collaborate.
I would like to build Own LLM and share it freely.
aritradhuan@vivobook:~$ nano manifesto.txt
```

manifesto.txt file (opened with nano)

```
  GNU nano 8.4                                                    manifesto.txt
On 29 March 2026, I believe open source is about Everything.
Tools like Python help developers collaborate.
I would like to build Own LLM and share it freely.
```

Explanation

This script takes user input and generates a text file. It uses input handling and file writing.

Conclusion

This project helped me understand the importance of open-source software in modern computing. By studying Git, I was able to learn not only how a widely used tool works, but also the philosophy and values behind its development. The origin of Git showed how real-world problems can lead to powerful open-source solutions, especially when developers aim for freedom and collaboration.

Through the Linux footprint section, I gained practical experience in installing and exploring Git within a Linux environment. This helped me understand how open-source tools interact with the system and how they are managed. Writing the shell scripts also improved my understanding of basic Bash concepts such as loops, conditions, and file handling.

In my opinion, open-source software plays a crucial role in encouraging innovation and knowledge sharing. Git is a strong example of how community-driven development can create tools that are reliable, flexible, and widely adopted. Overall, this project enhanced both my technical skills and my understanding of open-source principles, which will be useful in my future learning and development.

References

- Git Official Documentation — https://git-scm.com
- GNU General Public License (GPL v2) — https://www.gnu.org/licenses/gpl-2.0.html
- Open Source Initiative — https://opensource.org
- Linux Manual Pages — accessed using `man` command in terminal
- Ubuntu Package Management Documentation — https://ubuntu.com